

1. Variáveis, Tipos de Dados, Entrada e Saída, Operadores e Template Strings

Este material apresenta os **conceitos básicos da linguagem Python**, que servem como base para qualquer programa.

Sem o domínio desses fundamentos, estruturas mais avançadas como condicionais, laços e funções tornam-se difíceis de compreender.

▼ 1. Variáveis

Variáveis são usadas para **armazenar valores na memória do computador**, permitindo que esses valores sejam utilizados e modificados ao longo da execução do programa.

Em Python, uma variável é criada no momento em que recebe um valor.

```
nome = "Maria"  
idade = 25  
altura = 1.65
```

- `nome` armazena um texto.
- `idade` armazena um número inteiro.
- `altura` armazena um número decimal.

Uma forma simples de entender variáveis é imaginá-las como **caixas com nomes**, onde cada caixa guarda uma informação.

▼ 2. Regras para Definir Variáveis em Python

Python possui regras **obrigatórias** para nomes de variáveis.

Caso alguma delas seja violada, o código gera erro e não é executado.

Regras Obrigatórias

1. Não pode começar com número

```
2nome = "Ana"    # inválido
nome2 = "Ana"    # válido
```

2. Pode conter apenas letras, números e underline (_)

Caracteres especiais não são permitidos.

```
nome_usuario = "Carlos" # válido
nome-usuario = "Carlos" # inválido
```

3. Não pode conter espaços

```
nome completo = "Maria" # inválido
nome_completo = "Maria" # válido
```

4. Não pode usar palavras reservadas da linguagem

```
class = "Python" # inválido
for = 10         # inválido
```

Algumas palavras reservadas do Python:

if , else , for , while , class , def , return , True , False , None

▼ 3. Padrão de Nomenclatura em Python

Padrão oficial da linguagem

👉 **snake_case** é o padrão oficial do Python.

Esse padrão define que:

- Os nomes devem ser escritos em letras minúsculas
- As palavras devem ser separadas por underline (_)

```
nome_completo = "Maria Silva"
media_final = 8.5
```

```
total_alunos = 30
```

Esse padrão é definido pela **PEP 8**, o guia de estilo oficial do Python.

Outros padrões de escrita

camelCase (não recomendado para variáveis em Python)

Muito comum em JavaScript e Java.

```
nomeCompleto = "Maria"
```

Funciona, mas **não segue o padrão da linguagem Python**.

PascalCase

```
NomeCompleto = "Maria"
```

Em Python, esse padrão é utilizado **apenas para nomes de classes**, não para variáveis.

▼ 4. Constantes em Python

Python não possui constantes reais, mas existe uma **convenção amplamente aceita**.

Variáveis escritas em letras maiúsculas indicam que o valor **não deve ser alterado**.

```
PI = 3.14159  
TAXA_JUROS = 0.05
```

Essa regra é apenas uma convenção de escrita, não uma imposição do interpretador.

▼ 5. Boas Práticas ao Nomear Variáveis

- Usar nomes claros e descritivos
- Evitar abreviações confusas
- Não usar letras únicas sem contexto (`x`, `y`, `z`)

- Evitar nomes genéricos demais

Exemplos ruins

```
a = 10  
x1 = 50  
valor123 = 8
```

Exemplos bons

```
idade_usuario = 10  
quantidade_produtos = 50  
nota_final = 8
```

▼ 6. Tipagem Dinâmica em Python

Python é uma linguagem **dinamicamente tipada**.

Isso significa que:

- Não é necessário declarar o tipo da variável
- O interpretador identifica automaticamente o tipo do dado
- Uma mesma variável pode mudar de tipo durante a execução

```
x = 10  
x = "dez"  
x = True
```

Essa característica oferece flexibilidade, mas exige atenção para evitar erros lógicos em programas maiores.

▼ 7. Tipos de Dados em Python

Python possui diversos tipos de dados nativos. Os mais comuns são:

Inteiro (`int`)

```
idade = 42  
dia = 31
```

```
print(type(idade))  
print(type(dia))
```

Ponto Flutuante (**float**)

```
peso = 68.8  
altura = 1.74  
  
print(type(peso))  
print(type(altura))
```

Complexo (**complex**)

```
a = 12 + 8j  
b = 6 + 90j  
  
print(type(a))  
print(type(b))
```

String (**str**)

```
nome = 'Guilherme'  
sobrenome = "Henrique"  
lugar = "mc donald's"  
  
print(type(nome))  
print(type(sobrenome))
```

Strings podem ser delimitadas por aspas simples ou duplas.

Booleano (**bool**)

```
checked = True  
not_checked = False
```

```
print(type(checkered))
print(type(not_checked))
```

Internamente:

- `True` equivale a `1`
- `False` equivale a `0`

Lista (`list`) (*conceito introdutório*)

```
alunos = ['Ana', 'Bruna', 'Davi']
notas = [8.5, 9, 7.5]
```

Dicionário (`dict`) (*conceito introdutório*)

```
usuarios = {'Alice': 12, 'Mário': 24, 'Leo': 32}
```

Tupla (`tuple`) (*conceito introdutório*)

```
valores = (120, 50, 44, 91)
```

Tuplas são **imutáveis**. Tentativas de modificação geram erro.

▼ 8. Conversão de Tipos (Casting)

Em algumas situações, é necessário alterar o tipo de um valor.

```
int("10")
float("3.5")
str(100)
```

Conversões incorretas geram erro:

```
int("abc") # ValueError
```

▼ 9. Entrada e Saída de Dados

Entrada de Dados

A função `input()` recebe dados do usuário.

O valor retornado é sempre do tipo `str`.

```
nome = input("Digite um nome: ")
```

Saída de Dados

A função `print()` exibe informações na tela.

```
print(nome)
```

▼ 10. Template Strings (Formatação de Texto)

A formatação de strings permite inserir valores dentro de textos.

Operador `%` (forma antiga)

```
print("Olá, %s. Você tem %s." % ("Jose", 18))
```

Método `.format()`

```
print('O {} {} e {}'.format('carro', 'amarelo', 'rápido'))
```

f-strings (forma recomendada)

```
name = "José"
print(f"Olá {name}, prazer em conhecê-lo!")
```

Sempre que possível, prefira **f-strings** por clareza e legibilidade.

▼ 11. Operadores em Python

Os operadores permitem realizar cálculos, comparações e decisões.

Operadores Aritméticos

Operador	Operação
<code>+</code>	Soma
<code>-</code>	Subtração
<code>*</code>	Multiplicação
<code>/</code>	Divisão
<code>//</code>	Divisão inteira
<code>%</code>	Resto
<code>**</code>	Potência

Operadores de Comparação

Operador	Significado
<code>==</code>	Igual
<code>!=</code>	Diferente
<code>></code>	Maior
<code><</code>	Menor
<code>>=</code>	Maior ou igual
<code><=</code>	Menor ou igual

Operadores de Atribuição

Operador	Exemplo
<code>=</code>	<code>x = 5</code>
<code>+=</code>	<code>x += 1</code>
<code>-=</code>	<code>x -= 1</code>
<code>*=</code>	<code>x *= 2</code>

Operadores Lógicos

Operador	Significado
<code>and</code>	E
<code>or</code>	Ou

Operador	Significado
<code>not</code>	Não

Operadores de Identidade

Operador	Uso
<code>is</code>	Mesmo objeto
<code>is not</code>	Objeto diferente

Operadores de Associação

Operador	Uso
<code>in</code>	Pertence
<code>not in</code>	Não pertence

▼ 12. Resumo Geral

- Variáveis armazenam dados na memória.
- Python é dinamicamente tipado.
- Existem vários tipos de dados nativos.
- O padrão oficial de nomenclatura é **snake_case**.
- `input()` recebe dados do usuário.
- `print()` exibe informações.
- f-strings são a forma recomendada de formatação.
- Operadores permitem cálculos, comparações e decisões.

▼ Leituras Recomendadas

- PEP 8. Guia de Estilo do Python
<https://peps.python.org/pep-0008/>
- Documentação Oficial. Tipos de Dados
<https://docs.python.org/pt-br/3/library/stdtypes.html>
- Python Academy. Operadores em Python

<https://pythonacademy.com.br/blog/operadores-aritmeticos-e-logicos-em-python>

- Real Python. Variables and Data Types

<https://realpython.com/python-variables/>

▼ Observação crítica, como colega

A maioria dos problemas iniciais em Python não vem da sintaxe, mas de **nomenclatura ruim, tipos mal compreendidos e conversões implícitas**. Esse material resolve exatamente esses pontos.